planning - ex3

Omer Aviram - 206061319 , Itamar Hadida - 206438509

May 2026

3. Hyperparameters Used

- **General:** $\gamma = 0.99$, Total steps: 500,000, Evaluation interval: 10,000.
- **Q-Learning & SARSA:** Learning rate (α) = 0.1, Exploration (ϵ) decay from 1.0 to 0.05.
- **REINFORCE:** Policy LR = 0.005, Value LR = 0.05, Baseline = True, Entropy coefficient = 0.01.

4. Explanation of How $V(s_0)$ is Evaluated

Every 10,000 steps, the training process is paused. The agent plays 500 episodes using a **strictly greedy policy** (no exploration or randomness).
Specifically, for Q-learning and SARSA, the agent always picks the action with the highest Q-value for a given state (`greedy_action`).
For REINFORCE, it picks the action with the highest probability from the softmax policy.
The value of $V(s_0)$ is the average of the total returns from these 500 evaluation episodes.

5. Discussion of Results

- **Performance:** Q-learning and SARSA successfully learned the environment, reaching a value of approximately 0.25. REINFORCE completely failed, remaining close to 0 throughout the entire training process.
- **Q-learning vs. SARSA:** Q-learning reached slightly higher values because it is an **off-policy** algorithm (it assumes the agent will take the optimal future actions). SARSA is an **on-policy** algorithm and is therefore more conservative—it accounts for the mistakes the agent might make while exploring the dangerous environment, leading to slightly lower state evaluations.
- **Sparse Rewards & Stochasticity (Slippery Ice):** The combination of a sparse reward structure (receiving +1 only at the goal) and high stochasticity is the primary reason for REINFORCE's failure. Since it is a **Monte Carlo** method, the random slips accumulate over the entire episode, creating massive variance. Without intermediate rewards to guide the gradient, this variance completely destroys the learning signal. In contrast, Q-learning and SARSA use **Temporal Difference** (a 1-step lookahead). As soon as they randomly stumble upon the goal once, they bootstrap that value backward step-by-step, isolating the randomness and enabling much more stable learning.